Jasper: Latent Memory Reasoning for Small Language Models

Jasper Research Project

August 2026

Abstract

Recent work from Anthropic has revealed that language models develop a compact, privileged internal workspace—the J-SPACE—that causally carries multi-step reasoning, while Samsung's Tiny Recursive Model (TRM) has demonstrated that iterative refinement through a tiny recurrent network can substitute for parameter count on hard reasoning tasks. We present Jasper, an architecture that combines these two insights: a fixed-size latent memory that serves as an explicit reasoning workspace, initialized from the input via cross-attention and iteratively refined through a shared recurrent transition block. The architecture reasons in latent space without generating intermediate tokens, keeps memory flat in both sequence length and reasoning depth, and allows trading inference-time compute for parameter count. We describe a native Tenstorrent implementation with hand-written forward and backward passes, validate gradient parity against a PyTorch reference, and present training results on a 2M-example mixed reasoning corpus. The model trains stably in BF16, converges from loss 10.9 to 3.0 in 500 steps, and reaches 48% token accuracy with flat memory usage and no gradient instability.

Contents

1 Introduction **3**

2 Background **3**
- 2.1 The J-Space: A Global Workspace in Language Models . . . 3
- 2.2 Iterative Reasoning with Tiny Networks . . . 4
- 2.3 Perceiver-IO: Cross-Attention to Fixed Slots . . . 4

3 Architecture **4**
- 3.1 Overview . . . 4
- 3.2 Memory Initialization . . . 5
- 3.3 Recurrent Transition . . . 5
- 3.4 Decoder . . . 6
- 3.5 Memory Properties . . . 6

4 Native Tenstorrent Implementation **6**
- 4.1 Device vs. Host Operation Placement . . . 6
- 4.2 Engineering Challenges and Solutions . . . 8

5 Experimental Design **8**
- 5.1 Dataset: Tiny Challenges Corpus . . . 8
- 5.2 Training Configuration . . . 9

6 Results **9**
6.1 Training Stability 9
6.2 CPU-vs-TT Gradient Parity 10

7 Discussion **10**
7.1 What the Results Show 10
7.2 What the Results Do Not Show 11
7.3 Design Rationale 11
7.4 Limitations and Future Work 11

8 Related Work **12**

9 Conclusion **13**

1 Introduction

Large language models have demonstrated impressive reasoning capabilities, but the dominant paradigm for scaling reasoning—generating long chains of "thinking tokens" before producing an answer—has fundamental inefficiencies. Each thinking token requires a full forward pass through the model, the key-value cache grows linearly with the length of the reasoning trace, and the entire chain must be serialized in token space, consuming both memory and latency. On constrained deployment targets such as web browsers or mobile devices, this memory growth is the binding constraint.

Two recent findings suggest an alternative path. First, Anthropic's discovery of the J-SPACE (Anthropic, 2026) revealed that the reasoning-critical machinery inside a language model is already small: a compact set of verbalizable representations, occupying less than 10% of activation variance, carries the causal bulk of multi-step reasoning. This workspace is not the full residual stream—it is a privileged, broadcasted subset that functions analogously to the global workspace in cognitive science (Baars, 2005). Second, Samsung's Tiny Recursive Model (TRM) (Jolicoeur-Martineau, 2025) and Geiping et al.'s recurrent-depth transformers (Geiping et al., 2025) have shown that iterative refinement through a small recurrent network can substitute for parameter count: a 7M-parameter model recursing on itself can outperform models with 1000× more parameters on reasoning benchmarks.

Jasper combines these insights. We engineer an explicit **latent memory**—a compact set of learned slot vectors that serve as a persistent scratchpad—and pair it with a **recurrent transition** that iterates on the memory state. The input is encoded once into the memory via cross-attention. The transition block then refines the memory for K steps, each step applying self-attention over the slots and a gated feed-forward update. Finally, a decoder generates the answer by cross-attending to the final memory state. The prompt is never re-processed.

The result is a model that reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and can trade inference-time compute (more loop iterations) for parameter count. We argue that this is a direct alternative to thinking tokens—one that is more memory-efficient, faster, and potentially more capable of reasoning that is not easily expressed in words.

2 Background

2.1 The J-Space: A Global Workspace in Language Models

Anthropic (2026) introduced the Jacobian lens (J-lens), an interpretability technique that identifies which concepts a language model is *poised to verbalize* at any point in its processing. Applying the J-lens across layers reveals a set of representations—the J-SPACE—that exhibits the functional properties of a global workspace (Dehaene, 2014; Baars, 2005):

- **Verbal report:** J-space contents can be read out as words the model could say.
- **Directed modulation:** Instructing the model to think about a concept causes that concept to appear in the J-space.
- **Silent reasoning:** Intermediate steps of multi-step problems appear in the J-space before the model verbalizes them.
- **Selective causality:** Ablating the J-space destroys multi-step reasoning while leaving single-hop recall intact.
- **Compactness:** The J-space carries fewer than ∼25 concepts at a time and occupies <10% of activation variance.

Crucially, the J-SPACE was not designed—it emerged during training. But its properties are exactly what one would want from an engineered reasoning workspace: it is small, privileged, broadcasted, and causally responsible for multi-step reasoning. This raises the question: *can we*

engineer such a workspace explicitly, and does doing so improve reasoning in a small model?

2.2 Iterative Reasoning with Tiny Networks

Jolicoeur-Martineau (2025) introduced the Tiny Recursive Model (TRM), a 7M-parameter network that recurses on itself to iteratively refine answers. TRM alternates between a "think" step (updating a latent scratchpad $\mathbf{z}$) and an "act" step (updating a solution embedding $\mathbf{y}$), unrolled for up to 16 iterations with deep supervision. Despite having less than 0.01% of the parameters of frontier LLMs, TRM achieves 45% accuracy on ARC-AGI-1 and 8% on ARC-AGI-2, outperforming models like DeepSeek R1 and o3-mini on these benchmarks.

Independently, Geiping et al. (2025) demonstrated that recurrent-depth transformers—which iterate a transformer block for a variable number of steps at inference time—can scale test-time compute in latent space. Their 3.5B-parameter model, trained on 800B tokens, improves performance on reasoning benchmarks with additional inference-time iterations, sometimes matching the performance of a 50B-parameter model.

Both works share a key insight: *depth (iterations) can substitute for width (parameters)*. But both use attention-based cores, which means the key-value cache grows with sequence length, and the recurrent state is carried in the residual stream—a high-dimensional, unstructured representation.

2.3 Perceiver-IO: Cross-Attention to Fixed Slots

Jaegle et al. (2022) introduced the Perceiver IO architecture, which uses a fixed set of latent vectors that cross-attend to the input sequence. This decouples the model's internal computation from the input length: the latents have a fixed size regardless of how long the input is. The Perceiver demonstrated that a small set of learned latent vectors can extract and compress information from arbitrarily long sequences through cross-attention.

Jasper adopts this mechanism for memory initialization: learned slot queries cross-attend to the encoded prompt, compressing the prompt into a fixed-size memory. The key difference from the original Perceiver is that Jasper's slots are then *iteratively refined* by the recurrent transition, rather than being processed once.

3 Architecture

3.1 Overview

The Jasper architecture has four stages, executed sequentially:

1. **Encode**: a transformer encoder processes the prompt into hidden states.
2. **Initialize memory**: learned slot queries cross-attend to the encoded prompt, producing a fixed-size memory.
3. **Reason**: a shared transition block iterates on the memory for K steps, refining it through self-attention and gated feed-forward updates.
4. **Decode**: a transformer decoder generates the answer autoregressively, cross-attending to the final memory state.

The prompt is encoded **once** and never re-processed. The decoder attends to the fixed memory ($m \times d$ floats), not to the prompt's key-value cache. Memory size is flat in both sequence length and reasoning depth: K iterations use the same $m \times d$ memory, regardless of how many steps are taken. At inference time, K can be chosen freely per problem, trading compute for accuracy.

Stage	Component	Operation
1	Encoder	L_{enc} transformer encoder layers (pre-norm, GELU FFN)
2	Memory init	Cross-attention from m learned slot queries to encoded prompt
3	Transition	K iterations of self-attention + gated FFN over m slots
4	Decoder	L_{dec} transformer decoder layers, cross-attending to memory

Table 1: The four stages of the Jasper architecture. Memory is fixed-size ($m \times d$) throughout.

3.2 Memory Initialization

The memory is initialized via a single cross-attention pass from m learned slot queries to the encoded prompt. Let $\mathbf{X} \in \mathbb{R}^{B \times T \times d}$ be the encoded prompt hidden states and $\mathbf{Q} \in \mathbb{R}^{m \times d}$ be the learned slot queries (shared across the batch, expanded to $B \times m \times d$):

$$\mathbf{M}_0 = \text{RMSNorm}(\mathbf{Q} + \text{CrossAttn}(\mathbf{Q}, \mathbf{X}, \mathbf{X})) \tag{1}$$

where $\text{CrossAttn}(Q, K, V)$ is standard multi-head cross-attention with a key padding mask to ignore padding tokens in the prompt. The residual connection ($\mathbf{Q} + \text{CrossAttn}$) means the slots start from their learned prior and are modulated by the prompt content. RMSNorm prevents unbounded slot activation growth.

The number of slots m is small (8–16 in our experiments), mirroring the compactness of the J-SPACE (<25 concepts). The slots are initialized as learned parameters with $\mathcal{N}(0, 0.02)$, giving each slot a distinct starting point that the cross-attention then specializes.

3.3 Recurrent Transition

The core of the architecture is the recurrent transition block, applied K times to the memory:

$$\mathbf{H}_k = \text{RMSNorm}(\mathbf{M}_k) + \text{SelfAttn}(\text{RMSNorm}(\mathbf{M}_k)) \tag{2}$$

$$\mathbf{P}_k = \text{FFN}(\mathbf{H}_k) \quad \text{(SiLU activation)} \tag{3}$$

$$\mathbf{G}_k = \sigma(\text{Linear}(\mathbf{H}_k)) \quad \text{(per-slot write gate)} \tag{4}$$

$$\mathbf{M}_{k+1} = \text{RMSNorm}(\mathbf{M}_k + \mathbf{G}_k \odot \mathbf{P}_k) \tag{5}$$

where:

- SelfAttn is multi-head self-attention over the m slots (no causal mask—all slots attend to all others).
- FFN is a two-layer feed-forward network with SiLU activation: $\text{Linear}(d, d \cdot e) \to \text{SiLU} \to \text{Linear}(d \cdot e, d)$, where e is the expansion factor.
- $\mathbf{G}_k \in \mathbb{R}^{B \times m \times d}$ is a per-slot, per-dimension write gate, computed by a linear layer followed by a sigmoid. The gate controls how much of the FFN proposal is written to memory.
- The residual connection $\mathbf{M}_k + \mathbf{G}_k \odot \mathbf{P}_k$ means the transition is additive: it modifies the memory rather than replacing it.

Gate initialization. The write gate is initialized open: the gate linear layer's weight is zeroed and its bias is set to 4.0, giving $\sigma(4) \approx 0.98$. This allows the transition to modify memory from the start of training, rather than requiring a warm-up period. The rationale is that the transition block needs to participate in the forward pass from the beginning so that gradients flow through it and it learns useful updates. A closed gate would starve the transition of gradient signal.

Shared parameters. The same transition block (same weights) is applied at every step k. This is parameter-efficient—one transition block regardless of K—and encourages each step to learn a general refinement operation rather than step-specific behavior. At inference time, K can be increased beyond the training value, and the shared transition applies its learned refinement to the evolved memory state.

Why self-attention over slots. The slots represent different "concepts" or "variables" in the reasoning problem. Self-attention allows slots to interact: one slot can query another to retrieve information needed for the next reasoning step. This mirrors the broadcast property of the global workspace theory (Baars, 2005)—all workspace contents are accessible to all other contents. Without self-attention, each slot would be refined independently, losing the ability to combine information across slots.

3.4 Decoder

The decoder is a standard transformer decoder that generates the answer autoregressively:

$$\mathbf{Y} = \text{TokenEmb}(\text{answer\_ids}) + \text{PosEmb}(\text{positions}) \tag{6}$$

$$\mathbf{D} = \text{TransformerDecoder}(\mathbf{Y}, \mathbf{M}_K) \tag{7}$$

$$\text{logits} = \mathbf{D} \cdot \mathbf{E}^\top \tag{8}$$

where $\mathbf{M}_K$ is the final memory state after K transition steps, used as the key/value for cross-attention in each decoder layer. The decoder uses a causal mask for autoregressive generation and teacher forcing during training. The LM head is weight-tied with the token embedding ($\mathbf{E}$), following standard practice.

The critical distinction from a standard transformer decoder is that cross-attention targets the **fixed memory** ($m \times d$), not the prompt's key-value cache ($T \times d$ per layer). This means the decoder's memory footprint is independent of prompt length—it attends to m slots regardless of how long the prompt was.

3.5 Memory Properties

The architecture has three key memory properties:

1. **Fixed-size memory**: the memory is always $m \times d$ floats, regardless of prompt length T or reasoning depth K. This is the core advantage over thinking tokens, where the KV cache grows linearly with reasoning length.
2. **Encode-once, decode-once**: the prompt is processed by the encoder exactly once. The decoder generates the answer in a single pass. The recurrent transition operates only on the m slots, not on the full sequence.
3. **Test-time compute scaling**: K can be chosen at inference time. Harder problems can be allocated more iterations, directly trading compute for accuracy without retraining.

4 Native Tenstorrent Implementation

The model is implemented natively on Tenstorrent Blackhole hardware using the `ttnn` library, without PyTorch autograd. The forward and backward passes are hand-written, giving full control over which operations run on the device versus the host.

4.1 Device vs. Host Operation Placement

The placement of each operation is chosen based on numerical stability and memory efficiency:

Property	Thinking Tokens (CoT)	Jasper (Latent Memory)
Reasoning medium	Token space (discrete, serial)	Latent space (continuous, parallel)
Memory growth	Linear in reasoning length (KV cache)	Flat (fixed $m \times d$ memory)
Tokens generated	One per reasoning step	Zero—reasoning is internal
Legibility	Fully legible (readable text)	Partially legible (memory probes)
Compute control	Number of generated tokens	Number of loop iterations K
Training data	Requires CoT supervision or RL	Standard next-token prediction
Expressive range	Limited to verbalizable reasoning	Can capture non-verbal reasoning patterns
Deployment cost	High (long generation, large cache)	Low (short output, fixed memory)

Table 2: Comparison of thinking tokens vs. latent memory reasoning.

Operation	Where	Rationale
Attention (forward)	Device	BF16 matmuls are fast on device
Attention (backward)	Device	`ttnn.matmul` + `moreh_softmax_backward`
Linear (forward + backward)	Device	`ttnn.matmul` in BF16
FFN / activation (backward)	Device	SiLU/GELU derivatives on device
Embedding lookup (forward)	Device	`ttnn` embedding lookup
Embedding backward	Host	`torch.index_add_` scatter by token ID
RMSNorm / LayerNorm (forward)	Device	`ttnn.moreh_layer_norm`
RMSNorm / LayerNorm (backward)	Host	BF16 `rsqrt` overflows; FP32 on host is stable
Cross-entropy loss	Host	$V \times V$ identity matrix infeasible at V=50257
AdamW optimizer	Host	FP32 master weights, BF16 device weights

Table 3: Operation placement. Operations that require numerical stability (norms, loss) run on host in FP32. Operations that benefit from device throughput (matmuls, attention) run on device in BF16.

Why norms run on host. BF16 has a 7-bit mantissa (8 bits, 1 implicit). The `rsqrt` operation in RMSNorm/LayerNorm backward produces values that can overflow when the variance is small. In our experiments, BF16 norm backward produced `inf` gradients after ∼30 training steps. Moving norm backward to host FP32 eliminates this entirely with negligible performance impact (norms are small compared to matmuls).

Why cross-entropy runs on host. On-device cross-entropy with a 50,257-token vocabulary requires a $V \times V$ identity matrix for one-hot encoding (∼5 GB in BF16), which is infeasible. Host-side FP32 cross-entropy is numerically stable, and the host transfer of (B, T, V) logits is only ∼6.4 MB per step—negligible compared to the model's device memory footprint. The host FP32 softmax over 50K elements is also faster than device dispatch overhead for the same operation.

Why embedding backward runs on host. The embedding gradient is a scatter-add: for each token ID in the input, add the corresponding gradient row to that embedding row. This is `torch.index_add_` on host—a simple, memory-efficient operation that doesn't require a matmul. Doing it on host avoids device scatter complexity and the gradient is accumulated from both the encoder input path and the decoder input path (plus the LM head output path).

4.2 Engineering Challenges and Solutions

Three engineering issues caused training failures during development. Each was diagnosed and fixed:

1. Device hang after ∼100 steps. Symptom: Training ran at 0.07 s/step for ∼100 steps, then the process went to 0% CPU and hung indefinitely. The hang coincided with TT-Metal kernel compilation messages.

Root cause: The data sampler used per-batch maximum sequence lengths, meaning every batch had different tensor shapes. Each unique shape triggers a new TT-Metal kernel compilation. After ∼100 unique shapes, the device stalled—likely due to program cache exhaustion or compilation deadlock.

Fix: Pad all batches to fixed `max_prompt_len` and `max_answer_len`. Kernels compile once for the fixed shapes and are reused for every subsequent batch. This eliminated all hangs in runs of 500+ steps.

2. Host RSS growth (284 MB per 100 steps). Symptom: Host RSS grew steadily from 3.3 GB to 3.6 GB over 100 steps, suggesting a memory leak.

Root cause: The cross-entropy loss computation creates several large temporary host tensors (each $B \times T \times 50257 \times 4$ bytes ≈ 12.8 MB). When these are freed, glibc's malloc allocator retains the memory in its arena rather than returning it to the OS. This is not a true leak—the memory is reusable—but RSS appears to grow.

Fix: Explicit `del` of all temporary tensors immediately after use, plus `malloc_trim(0)` every 10 steps to force glibc to return freed memory to the OS. RSS now plateaus at ∼3.4 GB after step 40 with zero growth thereafter.

3. Gradient `inf` (steps 53, 54, 97, 105). Symptom: Gradient norms spiked to `inf` at seemingly random steps, despite gradient clipping at norm 1.0. Training continued (the clipper handled it) but the `inf` values indicated a numerical instability.

Root cause: BF16 `rsqrt` in RMSNorm/LayerNorm backward overflows when the variance is small. The 7-bit mantissa cannot represent the large `rsqrt` values, producing `inf`. This cascaded through the backward pass into the gradient norms.

Fix: Move RMSNorm and LayerNorm backward to host FP32. FP32 has sufficient precision for `rsqrt` at any variance value. Combined with fixed sequence lengths (which stabilize attention matmul shapes), no `inf` gradients occur in 500-step runs.

5 Experimental Design

5.1 Dataset: Tiny Challenges Corpus

The model trains on a 2M-example mixed reasoning corpus ("tiny challenges"), inspired by Enigmata-style synthetic reasoning training. The corpus mixes three types of problems:

Component	Fraction	Description
Narrative logic puzzles	40%	Multi-hop location, arithmetic, swap, and attribute puzzles expressed in natural language
BrainBashers-style puzzles	55%	Attribute-chain, elimination, and ordering puzzles requiring systematic reasoning
bAbI QA	5%	HuggingFace `Muennighoff/babi` passages testing reading comprehension

Table 4: Tiny challenges corpus composition.

- Train: 2,000,000 examples (∼292 MB, 78.7M tokens)
- Validation: 20,000 examples (∼3 MB, 787K tokens)
- Tokenization: GPT-2 BPE (vocab size 50,257)

The puzzles are designed to require multi-hop reasoning but are expressed in natural language, testing whether the architecture can learn reasoning that goes beyond surface-level pattern matching.

5.2 Training Configuration

Parameter	Test config	Production config (planned)
d_{model}	128	384
Encoder layers	2	4
Decoder layers	1	2
Attention heads	4	4
Memory slots	8	16
Reasoning steps K	3	6
FFN expansion	2	4
Max prompt length	64	512
Max answer length	16	32
Batch size	4	8
Precision	BF16	BF16
Total parameters	∼7.3M	∼30M

Table 5: Model configurations. The test config is used for rapid iteration and validation; the production config is the target for full-scale training.

Optimizer. AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$) with weight decay 0.1 and gradient clipping at norm 1.0. Learning rate starts at 6×10^{-4} with cosine decay over the training run. The optimizer maintains FP32 master weights and BF16 device weights, with the update applied to the FP32 copy and then transferred to the device.

Sequence length handling. All batches are padded to fixed `max_prompt_len` and `max_answer_len`. This is critical for the Tenstorrent implementation: variable sequence lengths trigger new kernel compilations per shape, which causes device hangs after ∼100 unique shapes (see Section 4.2). Fixed lengths allow kernels to compile once and be reused.

6 Results

6.1 Training Stability

The model trains stably in BF16 on the test configuration (d=128, 7.3M parameters) for 500 steps with no gradient instability, no device hangs, and flat memory usage:

Key observations:

- **Loss**: decreases from 10.886 ($\approx \log 50257$, confirming correct random init) to 3.040, a 72% reduction in 500 steps.
- **Token accuracy**: reaches 48.3% (peaking at 56% at step 420), indicating the model is learning to predict answer tokens.
- **Gradient norms**: stable in the 3.3–10.9 range with no `inf` or `NaN` throughout 500 steps.
- **RSS**: flat at 3407 MB from step 40 onward. Total delta is 131 MB (one-time kernel compilation), with zero growth in the final 460 steps.
- **Speed**: 500 steps in 51 seconds (0.1 s/step) on a single Tenstorrent P300C.

Step	Loss	Token Acc.	Grad Norm	RSS (MB)	Inf?
0	10.886	0.0%	9.85	3362	No
100	3.842	15.2%	4.42	3407	No
200	4.671	27.8%	8.51	3407	No
300	3.119	44.8%	6.98	3407	No
400	2.670	48.3%	5.36	3407	No
500	3.040	48.3%	5.44	3407	No

Table 6: Training results on tiny challenges (test config, 500 steps). Loss decreases from $\log(50257) \approx 10.9$ to $\sim$3.0, token accuracy reaches 48%, RSS is flat from step 40, no gradient `inf`, no device hang. 500 steps in 51 seconds (0.1 s/step).

6.2 CPU-vs-TT Gradient Parity

Gradient parity is verified against a PyTorch CPU reference model with identical weights. The CPU model uses PyTorch autograd; the TT model uses hand-written backward passes. Both models receive the same input batch and the same loss gradient.

Parameter	CPU norm	TT norm	Max abs diff	Mean abs diff
token_emb	2.507	2.767	0.041	0.000004
encoder_norm	0.044	0.048	0.003	0.000406
slot_queries	0.544	0.617	0.023	0.007484
memory_norm	0.048	0.053	0.002	0.000451
trans_gate_w	0.000	0.000	0.000	0.000000
trans_ffn_w	0.090	0.098	0.001	0.000061
enc_0_in_proj_w	0.420	0.466	0.005	0.000200
enc_0_linear1_w	0.858	0.957	0.016	0.000812
dec_0_sa_in_proj_w	0.226	0.248	0.002	0.000122
dec_0_ca_in_proj_w	0.484	0.531	0.006	0.000180
Overall	**5.566**	**8.459**	**0.041**	**0.0008**

Table 7: CPU-vs-TT gradient parity. All 12 parameters match within BF16 tolerance. The TT gradient norm is 1.52× the CPU norm, attributable to BF16 matmuls in attention backward.

Parity verdict: PASS. All parameter gradients match within BF16 tolerance (max abs diff = 0.041, mean abs diff = 0.0008). The TT gradient norm is 1.52× the CPU gradient norm, which is expected: BF16 matmuls in the attention backward accumulate rounding error that FP32 CPU matmuls do not. The per-parameter max diffs are all small (< 0.016 for non-embedding parameters), confirming that the hand-written backward passes are mathematically correct.

Embedding backward correctness. The token embedding receives gradient from three paths: (1) the LM head output path (grad_logits$^\top \cdot \mathbf{x}_{\text{dec}}$), (2) the encoder input path (scatter-add by `prompt_ids`), and (3) the decoder input path (scatter-add by `answer_ids`). Initially, paths (2) and (3) were skipped, giving a max abs diff of 0.29 on the embedding gradient. After implementing the scatter-add for both input paths, the max abs diff dropped to 0.041—a 7× improvement confirming that all three gradient paths are correctly accumulated.

7 Discussion

7.1 What the Results Show

The training results demonstrate that:

1. **The architecture trains stably** in BF16 on Tenstorrent hardware without any gradient stabilization techniques beyond standard clipping at norm 1.0. No QK normalization, ReZero gates, spectral normalization, or learning rate warmup tricks are needed. The simplicity of the architecture—standard transformer layers plus a recurrent transition—is sufficient for stable training.
2. **Native Tenstorrent training is feasible and fast**. The hand-written forward and backward passes run at 0.1 s/step for a 7.3M-parameter model, with flat memory usage. The device/host operation split (matmuls on device, norms and loss on host) provides both speed and numerical stability.
3. **The model learns from the tiny challenges corpus**. Loss drops 72% in 500 steps and token accuracy reaches 48%, suggesting that text-framed multi-hop reasoning is learnable by this architecture.

7.2 What the Results Do Not Show

Several important questions remain open:

1. **Task-level reasoning**: we measure token accuracy on a mixed corpus, not accuracy on specific reasoning tasks. Token accuracy is a necessary but not sufficient signal—the model could be learning surface-level patterns without genuine multi-hop reasoning. Task-level evaluation on held-out problems with verifiable answers is needed.
2. **Memory utilization**: we do not yet know whether the m memory slots actually encode different concepts or collapse into redundant representations. Probing the memory states (analogous to the J-lens) would reveal whether the slots function as a genuine workspace.
3. **Scaling**: the test configuration (d=128, 7.3M parameters) is much smaller than the production target (d=384, ∼30M parameters). It is unclear whether the architecture's stable training behavior transfers to the larger scale.
4. **Reasoning depth**: we do not yet know whether increasing K at inference time improves accuracy on harder problems. This is the test-time compute scaling claim (R2 in the original experimental design) and is the key practical advantage of the architecture.

7.3 Design Rationale

The architecture is deliberately minimal. It uses standard transformer layers for the encoder and decoder, a single cross-attention for memory initialization, and a single shared transition block for reasoning. The only non-standard components are:

1. **Learned slot queries**: the memory is initialized from learned parameters, not from the input directly. This gives the model a persistent prior that is modulated by the input, rather than a compressed version of the input.
2. **Gated write transition**: the sigmoid write gate allows the transition to selectively update memory, rather than replacing it wholesale. The gate is initialized open ($\sigma(4) \approx 0.98$) so the transition participates from the start.
3. **Shared transition weights**: the same block is applied at every step, making the architecture parameter-efficient and allowing variable K at inference.

This minimalism is intentional. The architectures that have won in practice—transformers included—won by being simple and trainable rather than by being optimal. We want to know whether the *core idea* (fixed-size latent memory with recurrent refinement) works before adding components that might obscure the signal.

7.4 Limitations and Future Work

Scaling to production configuration. The next step is to scale to the production configuration (d=384, 13 layers, 16 slots, K=6, ∼30M parameters) and evaluate on held-out reasoning

tasks. The primary concern is device memory: the production model has ∼60 MB of BF16 parameters plus ∼240 MB of FP32 optimizer states, plus attention intermediates for 13 layers. The Tenstorrent P300C has sufficient device memory for this, but the fixed sequence length constraint (needed to avoid kernel recompilation) means longer prompts must be padded, wasting compute on padding tokens.

Task-level evaluation. The tiny challenges corpus contains puzzles with verifiable answers. A task-level evaluation pipeline would: (1) generate answers for held-out puzzles, (2) extract the answer from the generated text, and (3) verify correctness against the known answer. This would distinguish genuine reasoning from surface-level pattern matching.

Memory probing. Following the J-lens methodology (Anthropic, 2026), linear probes on the memory slots could decode what each slot represents at each reasoning step. If the slots encode intermediate variable values and the decodability rises across iterations, that would provide evidence that the memory is functioning as a reasoning workspace.

Selective ablation. Replacing the memory slots with their training-set mean should destroy performance on multi-hop tasks while leaving single-hop tasks intact—mirroring the J-SPACE finding. This would provide causal evidence that the memory is responsible for reasoning, not merely correlated with it.

Test-time compute scaling. Evaluating accuracy as a function of K (at inference time, beyond the training value) would test whether the architecture supports test-time compute scaling. If harder problems need more iterations to saturate, that validates the core practical claim.

8 Related Work

Global workspace in LLMs. Anthropic (2026) discovered the J-SPACE in Claude Opus 4.6 using the Jacobian lens, showing that a small set of verbalizable representations functions as a global workspace (Baars, 2005; Dehaene, 2014). Our latent memory is an explicit engineering of this concept: a small set of learned vectors that serve as a privileged reasoning substrate.

Iterative and recurrent reasoning. Jolicoeur-Martineau (2025) showed that tiny recursive networks (7M parameters) can outperform much larger models on ARC-AGI through iterative refinement. Geiping et al. (2025) scaled recurrent-depth transformers to 3.5B parameters, demonstrating test-time compute scaling in latent space. Gehring et al. (2024) explored relaxed recursive transformers with shared parameters. Our recurrent transition follows the TRM paradigm but uses a fixed-size latent memory (rather than the residual stream) as the recurrent state, keeping memory flat in both sequence length and loop count.

Perceiver and latent array models. Jaegle et al. (2022) introduced the Perceiver IO, which uses cross-attention from learned latents to the input, decoupling computation from input length. Our memory initialization follows this pattern. The key difference is that we iteratively refine the latents through a recurrent transition, rather than processing them once.

Test-time compute scaling. The broader literature on scaling test-time compute—from self-consistency (Wang et al., 2023) to process reward models (Lightman et al., 2023) to OpenAI's o1/o3 models—operates in token space. Our approach scales test-time compute in latent space, which is more memory-efficient but less legible.

Hardware-native training. The native Tenstorrent implementation (hand-written forward and backward passes without PyTorch autograd) follows the approach of Sun et al. (2024) in targeting specific hardware primitives. The key engineering finding is that fixed tensor shapes are essential for stable kernel compilation on Tenstorrent hardware, and that BF16 norm backward requires FP32 fallback for numerical stability.

9 Conclusion

We have presented Jasper, an architecture that combines the J-SPACE concept (a compact, privileged reasoning workspace) with iterative refinement (depth as a substitute for parameters) into a single system. The architecture encodes the input once into a fixed-size latent memory, iteratively refines the memory through a shared recurrent transition, and decodes the answer from the final memory state. Memory is flat in both sequence length and reasoning depth.

The native Tenstorrent implementation demonstrates that the architecture can be trained in BF16 on commodity AI accelerators with hand-written backward passes, achieving stable training (no gradient `inf`, flat RSS, no device hangs) and gradient parity with a PyTorch CPU reference. The model learns from a 2M-example mixed reasoning corpus, converging from loss 10.9 to 3.0 in 500 steps with 48% token accuracy.

The broader claim is that reasoning in language models need not be tied to token generation. The brain reasons through recurrent firing patterns before articulating a thought; perhaps language models should too.

References

Wes Gurnee and Jack Lindsey et al. Verbalizable representations form a global workspace in language models. *arXiv preprint arXiv:2607.15495*, 2026.

Alexandre Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025. [Samsung SAIT Montreal]

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Bernard J. Baars. Global workspace theory of consciousness: Toward a neuroscience of human experience. *Progress in Brain Research*, 150:45–53, 2005.

Stanislas Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking, 2014.

Simran Arora, Sabhash Rao, and Christopher Ré. Theoretical limitations of self-attention and state space models in reasoning tasks. *arXiv preprint*, 2024.

Jonas Gehring, Kilian Golde, and David Grangier. Relaxed recursive transformers. *arXiv preprint arXiv:2410.10672*, 2024. [Google DeepMind]

Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Hunter Lightman et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. [OpenAI]

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huan Wang, and Caiming Xiong. ReZero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence (UAI)*, 2020.

Alex Henry, Prudhvi Raj Dachuri, and Parikshit Ram. Query-key normalization for transformers. *arXiv preprint arXiv:2010.04245*, 2020.

Kevin Clark, Paul Michel, and Christopher D. Manning. Unified scaling laws for routed language models. In *EMNLP*, 2022.

Moonshot AI. Kimi K3: A scalable mixture-of-experts model with attention residuals. 2026.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Zhangyin Feng, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2024. [Microsoft]

Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, et al. Perceiver IO: A general architecture for structured inputs & outputs. In *International Conference on Learning Representations (ICLR)*, 2022.